

目录

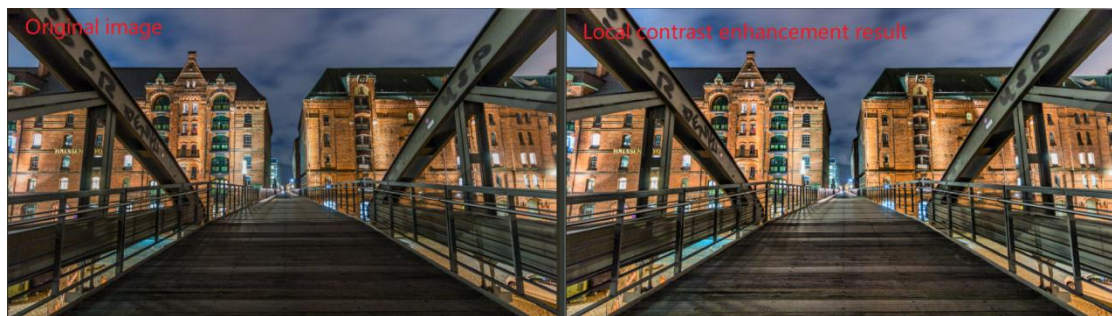
1. Introduction	3
2. Overall Framework Diagram and Data Flow.....	4
3. Detailed Description of Each Module	9
3.1 Luma Calculation	9
3.2 Luma Frequency Decomposition.....	10
3.3 3D LUT Slicing & Trilinear Interpolation	12
3.3.1 z index calculate.....	14
3.3.2 x index calculate	15
3.3.2 y index calculate	15
3.3.4 Trilinear Interpolation	16
3.4 Globla tone adjust	16
3.5 Luma Freq Recompose	18
3.6 Color Mapping.....	18
3.7 Dither	19
3.8 Block avg sd	20
3.9 Down Scale	23
3.10 Block avg luma.....	26
3.11 Block avg wopr	28
3.12 Content Detection	29
Weighted average	31
Detail calcu	31
content weight calcu	31
3.13 Local Histogram.....	32
3.14 ROI detection	35
3.15 Block avg texture	36

Revision Record

Revision	Date	description	Owner
0.7.1	2025/06/11	1. Add global tone adjust 2. Correct some write error 3. Delete CURVE_CONFIGURATION_COMPLETE registers(This registers is independent of the algorithm).	Chang Cao

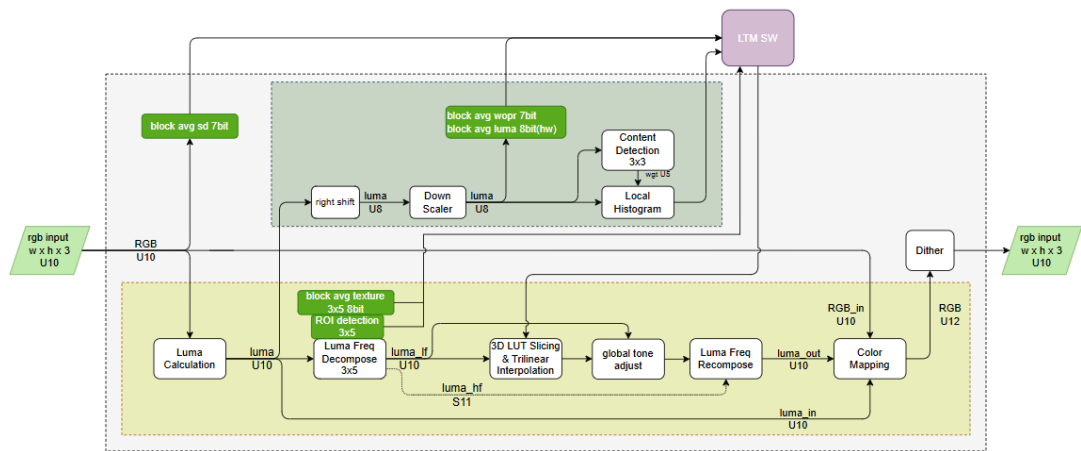
1. Introduction

LTM V2.0 is an integrated product combining local tone mapping(LTM)/local contrast enhancement, sunlight-readable enhancement(SRE), Fidelity optimized signal scaling(FOSS). This product features local contrast enhancement, sunlight visibility, and Low-power image display functionality. The local contrast enhancement improves image contrast, making details richer, while the sunlight-readable feature ensures clear screen visibility even in strong outdoor lighting conditions. Low-power image display reduces screen power consumption through image processing techniques. Results of LTM V2.0 are as follows:





2. Overall Framework Diagram and Data Flow



If ENABLE is 0, LTM V2.0 will disable. If ENABLE is 1, LTM V2.0 will do as the figure.

The following parameters are those configured by the software to the hardware

Name	Type	Default Value	Description
ENABLE	U1.0	1	Enable bit of LTMV2.0.
RGBY_COEFF[3]	U0.10	306, 601, 119	RGB to Y coefficients. [0, 1023)
GRAY_WEIGHT	U1.6	50	Weight of Y when blending between gray and lightness. 0: use lightness. 64: use gray. [0, 64]

LIGHTNESS_WEIGHT	U1.6	48	Lightness weight when blending between RGB min and RGB max. 0: RGB min, 64: RGB max.
ENABLE_FREQ_DECOMP	U1.0	True	Enable bit of luma frequency decomposition.
FREQ_DECOMP_COEFF_0	U8.0	0	Luma frequency decomposition filter coefficient 0.
FREQ_DECOMP_COEFF_1	U8.0	8	Luma frequency decomposition filter coefficient 1.
FREQ_DECOMP_COEFF_2	U8.0	16	Luma frequency decomposition filter coefficient 2.
FREQ_DECOMP_COEFF_3	U8.0	0	Luma frequency decomposition filter coefficient 3.
FREQ_DECOMP_COEFF_4	U8.0	16	Luma frequency decomposition filter coefficient 4.
FREQ_DECOMP_COEFF_5	U8.0	32	Luma frequency decomposition filter coefficient 5.
ENABLE_SLICE_AFFINE_GRID	U1.0	1	Enable bit of slice and affine.
ENABLE_HIST	U1.0	1	Enable bit of histogram statistics.
ENABLE_HIST_OVERLAP	U1.0	0	Enable bit of histogram overlap.
HIST_OVERLAP_RATIO	U0.10	0	Overlap ratio during histogram statistics.
HIST_GRID_DEPTH	U7.0	64	Histogram bin numbers.
GRID_DEPTH	U5.0 R/W	17	Affine grid depth.
GRID_HEIGHT	U4.0	8	Grid numbers in vertical direction.
GRID_WIDTH	U4.0	8	Grid numbers in horizontal direction.

GRID_Y_SCALE	U16.0		$((1 \ll 16) * \text{GRID_WIDTH}) / \text{ROI_X_LEN}$
GRID_X_SCALE	U16.0		$((1 \ll 16) * \text{GRID_HEIGHT}) / \text{ROI_Y_LEN}$
NON_ROI_MAPPING_WAY	U1.0	1	1: Non-ROI regions use the nearest block for mapping. 0: Non-ROI regions not mapping.
AFFINE_LUT[1088]	1088*U11.0		Affine LUT. This LUT should be programmed by LTM SW. 1088 entries = 8x8x17.
ENABLE_DITHER	U1.0	1	Enable bit of dither.
DITHER_TABLE_R[4]	4*U2.0	0,2,3,1	Dither threshold table of R
DITHER_TABLE_G[4]	4*U2.0	1,0,3,2	Dither threshold table of G
DITHER_TABLE_B[4]	4*U2.0	3,2,1,0	Dither threshold table of B
HIST_GRID_X_SCALE	U16.0		Norm coefficient of horizontal direction.
HIST_GRID_Y_SCALE	U16.0		Norm coefficient of vertical direction.
ENABLE_DS	U1.0	1	Enable bit of downscale.
DS_OUTPUT_WIDTH	U12.0		Down-scaler output width.
DS_OUTPUT_HEIGHT	U12.0		Down-scaler output height.
DS_INPUT_WIDTH	U12.0		Down-scaler input width..
DS_INPUT_HEIGHT	U12.0		Down-scaler input height.
ROI_X_START	U12.0	0	X-coordinate of ROI start point.
ROI_Y_START	U12.0	0	Y-coordinate of ROI start point.
ROI_X_END	U12.0		X-coordinate of ROI end point(last point of ROI).
ROI_Y_END	U12.0		Y-coordinate of ROI end point(last point of ROI).

ROI_X_LEN	U12.0		Width of ROI.
ROI_Y_LEN	U12.0		Height of ROI.
DS_HOR_SCALE_NO RM	U15.0		Norm coefficient of horizontal direction.
DS_VER_SCALE_NO RM	U15.0		Norm coefficient of vertical direction.
ENABLE_CONTENT_ DETECTION	U1.0	1	Enable bit of content detection
CONTENT_DETECTIO N_COEF0	U8.0	8	Content detection filter coefficient 0.
CONTENT_DETECTIO N_COEF1	U8.0	16	Content detection filter coefficient 1.
CONTENT_DETECTIO N_COEF2	U8.0	16	Content detection filter coefficient 2.
CONTENT_DETECTIO N_COEF3	U8.0	32	Content detection filter coefficient 3.
CONTENT_DETECTIO N_THRESHOLD0	U8.0	1	Content detection threshold coefficient 0.
CONTENT_DETECTIO N_THRESHOLD1	U8.0	12	Content detection threshold coefficient 1.
CONTENT_DETECTIO N_THRESHOLD2	U8.0	64	Content detection threshold coefficient 2.
CONTENT_DETECTIO N_THRESHOLD3	U8.0	128	Content detection threshold coefficient 3.
CONTENT_DETECTIO N_SLOPE0	U1.14	1489	Content detection slope coefficient 0.
CONTENT_DETECTIO N_SLOPE1	U1.14	256	Content detection slope coefficient 1.
CONTENT_DETECTIO	U2.0	2	Luma frequency decomposition filter

N_MIN_WGT			coefficient 5.
LUMA_BLENDING_ENABLE	U1.0	True	Enable bit of average luma calculate.
AD_TXT_JUD_EN	U1.0	True	Enable bit of average roi detect.
TEX_BLENDING_ENABLE	U1.0	True	Enable bit of average texture feature.
TEX_DIFF_LUT	33*U9.0		Lut use for texture feature calculate.
WOPR_BLENDING_ENABLE	U1.0	True	Enable bit of wopr module.
WOPR_THR	U8.0	230	Threshold of luma.
SD_BLENDING_ENABLE	U1.0	True	Enable bit of scan detection.
SD_R_THR	U8.0	95	R threshold of scan detection
SD_G_THR	U8.0	40	G threshold of scan detection
SD_B_THR	U8.0	20	B threshold of scan detection
SD_DELTA_MINMAX_THR	U8.0	15	Delta_rgb threshold of scan detection
SD_DELTA_RG_THR	U8.0	15	Abs_rg threshold of scan detection
BLOCK_RATIO_LR	U26.0		Norm coefficient of downscale resolution.
BLOCK_RATIO_HR	U26.0		Norm coefficient of original resolution.
USE_OUTPUT_LUMA	U1.0	True	Select use output luma or input luma lf.
TONE_ADJUST_LUT	129*U11.0	True	Enable bit of luma frequency decomposition.

Below are the parameters that the hardware needs to report to the software.

Name	Type	Description
HISTOGRAM	64*64*U32.0	The weight of pixels in histogram computation.

SD_RATIO	64*U7.0	Output block sd ratio.
LUMA_AVG	64*U8.0	Output average luma of each block.
WOPR_AVG	64*U7.0	Output white point ratio of each block.

Below are several functions that are commonly used in the following text:

1. **RoundShift**(val0, right_shift_bit), This function right-shifts val0 by right_shift_bit bits with rounding, as follows.

```

if val0 ≥ 0
    val0 += 1 << (right_shift_bit - 1)
else:
    val0 += (1 << (right_shift_bit - 1)) - 1

result = val0 >> right_shift_bit

```

2. **RoundMultiply**(val0, val1, right_shift_bit)

$\text{RoundMultiply}(\text{val0}, \text{val1}, \text{right_shift_bith}) = \text{RoundShift}(\text{val0} * \text{v1}, \text{right_shift_bit})$

3. **Clamp**(val, min, max)

$$\text{result} = \begin{cases} \text{min}, & \text{if } \text{val} < \text{min} \\ \text{max}, & \text{if } \text{val} > \text{max} \\ \text{val}, & \text{else} \end{cases}$$

4. **Lerp**(val0, val1, weight, weight_frac)

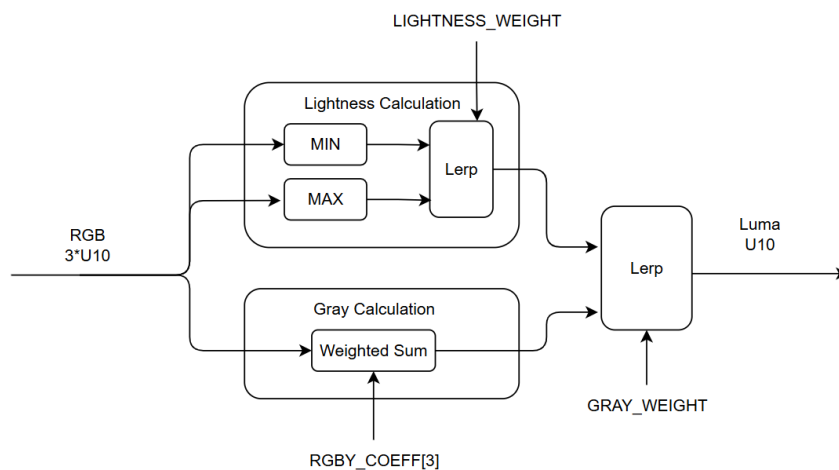
```

diff = val1 - val0
result = val0 + RoundShift(diff * weight, weight_frac)

```

3. Detailed Description of Each Module

3.1 Luma Calculation



Input

Name	Type	Description
rgb_input[3]	U10.0	Input RGB values.

Output

Name	Type	Description
luma	U10.0	Output luma values.

Registers

Name	Type	Default Value	Description
RGBY_COEFF[3]	U0.10	306, 601, 119	RGB to Y coefficients. [0, 1023)
GRAY_WEIGHT	U1.6	50	Weight of Y when blending between gray and lightness. 0: use lightness. 64: use gray.
LIGHTNESS_WEIGHT	U1.6	48	Lightness weight when blending between RGB min and RGB max. 0: RGB min, 64: RGB max.

This block calculates the luma value based on: (1) the weighted average of R/G/B values, (2) weighting of the maximum of R/G/B values and the minimum of R/G/B, or (3) the weighting of (1) and (2).

1. Gray_val calculate

```

weighted_r = RoundMultiply(RGBY_COEFF[0], r, 10)
weighted_g = RoundMultiply(RGBY_COEFF[1], g, 10)
weighted_b = RoundMultiply(RGBY_COEFF[2], b, 10)
gray_val = Clamp(weighted_r + weighted_g + weighted_b, 0, 1023)

```

2. Lightness calculate

```

min_val = min(r, g, b)
max_val = max(r, g, b)
lightness = Lerp(min_val, max_val, LIGHTNESS_WEIGHT, 6)

```

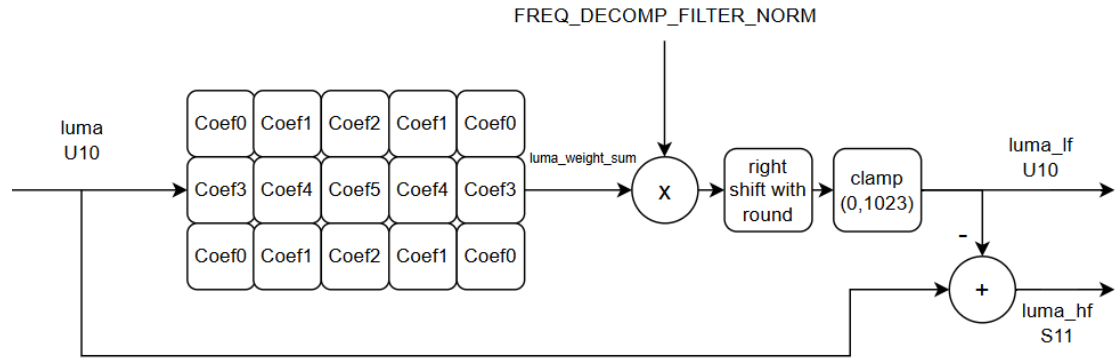
3. Luma calculate

```

luma = Clamp(Lerp(lightness, gray_val, GRAY_WEIGHT, 6), 0, 1023)

```

3.2 Luma Frequency Decomposition



This block separates the input luma image into a low-frequency component and a high-frequency component using 3x5 filter. The low-frequency component is enhanced by local tone mapping operators, while the high frequency component is kept untouched, and combined together at the Luma Frequency Recomposition block.

Input

Name	Type	Description
luma	U10.0	Input 3x5 luma values.

Output

Name	Type	Description
luma_lf	U10.0	Luma low-frequency values.
luma_hf	S11.0	Luma high-frequency values.

Registers

Name	Type	Default Value	Description
ENABLE_FREQ_DECOMP	U1.0	True	Enable bit of luma frequency decomposition.
FREQ_DECOMP_C_OEF_0	U8.0	0	Luma frequency decomposition filter coefficient 0.
FREQ_DECOMP_C_OEF_1	U8.0	8	Luma frequency decomposition filter coefficient 1.
FREQ_DECOMP_C_OEF_2	U8.0	16	Luma frequency decomposition filter coefficient 2.
FREQ_DECOMP_C	U8.0	0	Luma frequency decomposition filter

OEF_3			coefficient 3.
FREQ_DECOMP_C OEF_4	U8.0	16	Luma frequency decomposition filter coefficient 4.
FREQ_DECOMP_C OEF_5	U8.0	32	Luma frequency decomposition filter coefficient 5.

If the module is not enable(ENABLE_FREQ_DECOMP==0), luma_lf=luma, luma_hf=0. If the module is enable, The calculation process of this module is as follows.

$$\text{luma_weight_sum} = \text{SumMultiply}(\text{LUMA}, \text{COEF})$$

Where, SumMultiply first performs element-wise multiplication of the two input parameters (matrices), then sums up the resulting values.

LUMA refers to the luma values of pixels within a 3×5 window centered on the current point. Note that when indices in the following equation go out of bounds, clamping is required (square brackets in the equation denote clamping). For example, when i=0 and j=0, both i-2 and j-2 equal -2, which exceeds the minimum pixel position. These values should therefore be clamped to 0. Similarly, clamping is required when exceeding maximum pixel positions.

$$\text{LUMA} = \begin{pmatrix} \text{luma}_{[(i-1)(j-2)]} & \text{luma}_{[(i-1)(j-1)]} & \text{luma}_{[(i-1)j]} & \text{luma}_{[(i-1)(j+1)]} & \text{luma}_{[(i-1)(j+2)]} \\ \text{luma}_{[i(j-2)]} & \text{luma}_{[i(j-1)]} & \text{luma}_{[ij]} & \text{luma}_{[i(j+1)]} & \text{luma}_{[i(j+2)]} \\ \text{luma}_{[(i+1)(j-2)]} & \text{luma}_{[(i+1)(j-1)]} & \text{luma}_{[(i+1)j]} & \text{luma}_{[(i+1)(j+1)]} & \text{luma}_{[(i+1)(j+2)]} \end{pmatrix}$$

COEF is

$$\begin{pmatrix} \text{FREQ_DECOMP_COEF_0} & \text{FREQ_DECOMP_COEF_1} & \text{FREQ_DECOMP_COEF_2} & \text{FREQ_DECOMP_COEF_1} & \text{FREQ_DECOMP_COEF_0} \\ \text{FREQ_DECOMP_COEF_3} & \text{FREQ_DECOMP_COEF_4} & \text{FREQ_DECOMP_COEF_5} & \text{FREQ_DECOMP_COEF_4} & \text{FREQ_DECOMP_COEF_3} \\ \text{FREQ_DECOMP_COEF_0} & \text{FREQ_DECOMP_COEF_1} & \text{FREQ_DECOMP_COEF_2} & \text{FREQ_DECOMP_COEF_1} & \text{FREQ_DECOMP_COEF_0} \end{pmatrix}$$

The sum of the FREQ_DECOMP_COEF configured by the software algorithm is 128, so it can directly right shift luma_weight_sum to obtain the low-frequency component, as shown in the following formula:

$$\begin{aligned} \text{luma_lf} &= (\text{luma_weight_sum} + 64) \gg 7 \\ \text{luma_lf} &= \text{clamp}(\text{luma_lf}, 0, 1023) \\ \text{luma_hf} &= \text{luma} - \text{luma_lf} \end{aligned}$$

3.3 3D LUT Slicing & Trilinear Interpolation

If the module is not enable(ENABLE_SLICE_AFFINE_GRID==0), luma_lf_out = luma_lf_in, . If the module is enable, The calculation process of this module are as follows.

This block uses (1) low-frequency component of luma value calculated in luma freq decomposition module, and (2) the affine lut parameters from LTM SW to interpolate the out

luma for the current pixel. The interpolation uses trilinear interpolation from the 8 closest control points in the Affine Grid.

Input

Name	Type	Description
luma_lf_in	U10.0	Input luma values.

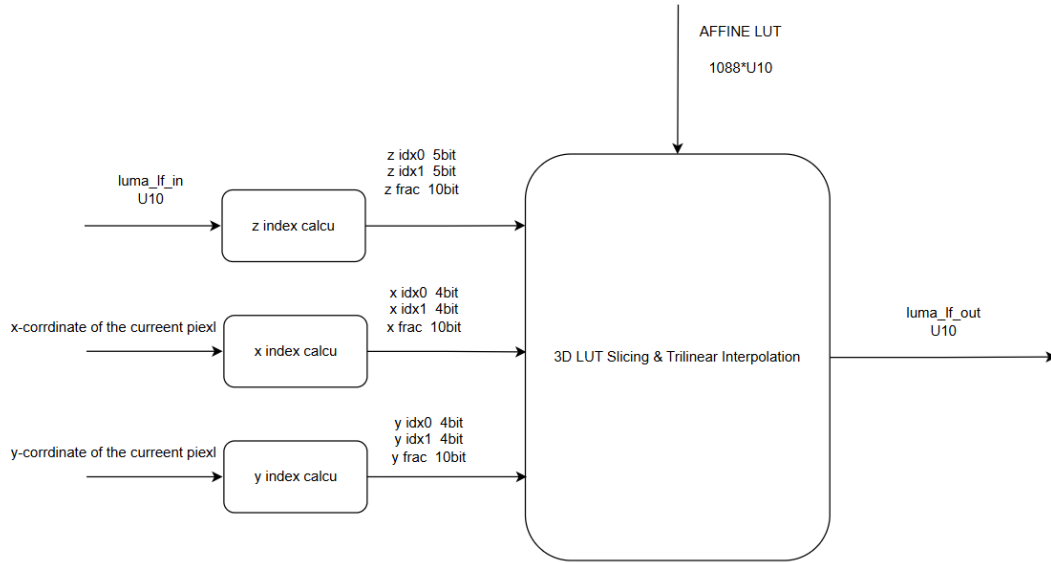
Output

Name	Type	Description
luma_lf_out	U10.0	Output luma values.

Registers

Name	Type	Default Value	Description
ENABLE_SLICE_AF FINE_GRID	U1.0	1	Enable bit of slice and affine.
NON_ROI_MAPPING _WAY	U1.0	1	1: Non-ROI regions use the nearest block for mapping. 0: Non-ROI regions not mapping.
GRID_DEPTH	U5.0	17	Affine grid depth.
GRID_HEIGHT	U4.0	8	Grid numbers in vertical direction.
GRID_WIDTH	U4.0	8	Grid numbers in horizontal direction.
AFFINE_LUT[1088]	U10.0		Affine LUT. This LUT should be programmed by LTM SW. 1088 entries = 17*8*8.
ROI_X_START	U12.0	0	X-coordinate of ROI start point.
ROI_Y_START	U12.0	0	Y-coordinate of ROI start point.
GRID_X_SCALE	U16.0		$((1 < 16) * \text{GRID_WIDTH}) / \text{ROI_X_LEN}$
GRID_Y_SCALE	U16.0		$((1 < 16) * \text{GRID_HEIGHT}) / \text{ROI_Y_LEN}$

The logic block diagram of this section is shown below:



When MAPPING_WAY is 0, only the region of interest (ROI) is mapped, while other regions will not mapping. SO as follows:

if NON_ROI_MAPPING_WAY == 0 && Non_ROI

$$\text{luma_lf_out} = \text{luma_lf_in}$$

where Non_ROI are

$$\begin{aligned}
 & ((x_corrinate \geq 0 \ \&\& \ x_corrinate \leq \text{ROI_X_START}) \\
 & \quad || \\
 & (x_corrinate > \text{ROI_X_END} \ \&\& \ x_corrinate \leq \text{width} - 1)) \\
 & \quad || \\
 & ((y_corrinate \geq 0 \ \&\& \ y_corrinate \leq \text{ROI_Y_START} - 1) \\
 & \quad || \\
 & (y_corrinate > \text{ROI_Y_END} \ \&\& \ y_corrinate \leq \text{height} - 1))
 \end{aligned}$$

Where width and height are width and height of image.

else, do as follows

3.3.1 z index calculate

This module calculates the depth position in the current affine LUT based on the luma_lf value, determining both the left and right reference points in the mapping curve.

$$\text{z_idx_l} = \text{luma_lf_in} \gg 6$$

$$\text{z_frac} = (\text{luma_lf_in} \& ((1 \ll 6) - 1)) \ll 4$$

$$\text{z_idx0} = \text{clamp}(\text{z_idx_l}, 0, \text{GRID_DEPTH} - 1)$$

$$\text{z_idx1} = \text{clamp}(\text{z_idx_l} + 1, 0, \text{GRID_DEPTH} - 1)$$

In the given equation, the number 6 represents log space of 1024/16, which 1024 represent the image contain 1024 gray level(10bit image) and 16 represent 16 segment of the mapping curve.

Left shift 4 of z_frac calculates is to adjust precision of z_frac to 10bit.

3.3.2 x index calculate

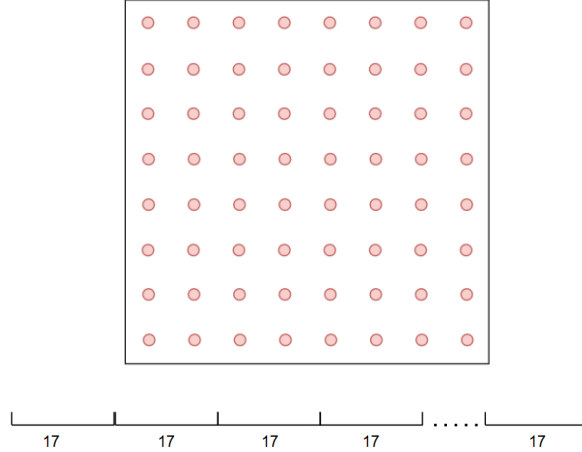
This module calculates the x position in the current affine LUT based on x corrdinate, determining the left and right mapping curves and their corresponding interpolation weights.

```

grid_x_scale_half = GRID_X_SCALE >> 1
x = x_corrdinate - ROI_X_START
scaled = (x * GRID_X_SCALE) + grid_x_scale_half - (1 << 15)
x_idx_l = scaled >> 16
x_frac = (scaled - (x_idx0 << 16)) >> 6
x_idx0 = clamp(x_idx_l, 0, GRID_WIDTH - 1)
x_idx1 = clamp(x_idx_l + 1, 0, GRID_WIDTH - 1)

```

where ROI_X_START , GRID_X_SCALE and GRID_WIDTH need software programme. GRID_X_SCALE equal to $(1 < 16) * \text{grid_width} / \text{roi_x_len}$, where grid_width is the grid nums in width direction. ROI_X_START is the x-coordinate of the ROI starting point.



The key formula among these calculations is the scaled computation. And the meaning of scaled is the current point's index of the blocks with 16bit precision. The floating-point representation of the formula is as follows:

$$\text{scaled_f} = (x + 0.5) * \frac{\text{grid_width}}{\text{roi_x_len}} - 0.5$$

The formula uses a center-aligned scaling method. For reference, please visit the following website: <https://www.cnblogs.com/jymg/p/11654554.html>.

3.3.2 y index calculate

This module calculates the y position in the current affine LUT based on the y corrdinate value, determining the up and botton mapping curves and their corresponding interpolation weights.

```

grid_y_scale_half = GRID_Y_SCALE >> 1
y = y_corrdinate - ROI_Y_START
scaled = (y * GRID_Y_SCALE) + grid_y_scale_half - (1 << 15)
y_idx_l = scaled >> 16
y_frac = (scaled - (y_idx0 << 16)) >> 6
y_idx0 = clamp(y_idx_l, 0, GRID_HEIGHT - 1)
y_idx1 = clamp(y_idx_l + 1, 0, GRID_HEIGHT - 1)

```

The calculation process of y index is similar to that of x index, and thus will not be elaborated here

3.3.4 Trilinear Interpolation

This module determines eight reference points in the affine LUT using the precomputed indices, then performs trilinear interpolation on these points by applying the calculated weighting factors to generate the final output

```

yi = {y_idx0, y_idx1, y_idx0, y_idx1, y_idx0, y_idx1, y_idx0, y_idx1}
xi = {x_idx0, x_idx0, x_idx1, x_idx1, x_idx0, x_idx0, x_idx1, x_idx1}
zi = {z_idx0, z_idx0, z_idx0, z_idx0, z_idx1, z_idx1, z_idx1, z_idx1}
lut[i] = affine_lut[z[i]] + (8 * 17) * y[i] + 17 * xi[i]  i = 0,1,2, ... ....7

```

where affine_lut is lut use to mapping luma_lf_in to luma_lf_out. The len of affine_lut is 1088, which represent 17*8*8.

```

lerp_y1 = Lerp(lut[0], lut[1], y_frac, 10)
lerp_y2 = Lerp(lut[2], lut[3], y_frac, 10)
lerp_y3 = Lerp(lut[4], lut[5], y_frac, 10)
lerp_y4 = Lerp(lut[6], lut[7], y_frac, 10)

lerp_x1 = Lerp(lerp_y1, lerp_y2, x_frac, 10)
lerp_x2 = Lerp(lerp_y3, lerp_y4, x_frac, 10)

lerp_z = Lerp(lerp_x1, lerp_x2, z_frac, 10)
luma_lf_out = clamp(Lerp_z, 0, 1023)

```

where lerp was defined in section 2 of this document.

3.4 Globla tone adjust

Input

Name	Type	Description
Luma lf out	U10.0	Output low-frequency luma after affine.

Luma lf	U10.0	Luma low-frequency values.
---------	-------	----------------------------

Output

Name	Type	Description
Luma lf out adjust	U10.0	Global tone adjust to luma lf out.

Registers

Name	Type	Default Value	Description
USE_OUTPUT_LUMA	U1.0	True	Select use output luma or input luma lf.
TONE_ADJUST_LUT	129*U11.0	True	Enable bit of luma frequency decomposition.

The function of this module is to perform global tone adjustment on the luma. When `USE_OUTPUT_LUMA == 1`, it adjusts the output low-frequency component luma. When `USE_OUTPUT_LUMA == 0`, it directly adjusts the input low-frequency component luma. **Let's take the case where `USE_OUTPUT_LUMA == 1` to explain the functionality of this module.**

$$\text{luma} = \begin{cases} \text{luma_lf_out}, & \text{if USE_OUTPUT_LUMA is True} \\ \text{luma_lf}, & \text{else} \end{cases}$$

If `NON_ROI_MAPPING_WAY == 0 && Non_ROI`

$$\text{luma_lf_out_adjust} = \text{luma}$$

where `Non_ROI` are

$$\begin{aligned} & ((x_corrinate \geq 0 \ \&\& \ x_corrinate \leq \text{ROI_X_START}) \\ & \quad || \\ & (x_corrinate > \text{ROI_X_END} \ \&\& \ x_corrinate \leq \text{width} - 1)) \\ & \quad || \\ & ((y_corrinate \geq 0 \ \&\& \ y_corrinate \leq \text{ROI_Y_START} - 1) \\ & \quad || \\ & (y_corrinate > \text{ROI_Y_END} \ \&\& \ y_corrinate \leq \text{height} - 1)) \end{aligned}$$

Where width and height are width and height of image.

else, do as follows

$$\begin{aligned} \text{idx0} &= (\text{luma} \gg 3) \\ \text{idx1} &= \text{clamp}(\text{idx0} + 1, 0, 128) \\ \text{wgt} &= \text{luma} \& ((1 \ll 3) - 1) \end{aligned}$$

```

val0 = TONE_ADJUST_LUT[idx0]
val1 = TONE_ADJUST_LUT[idx1]
luma_lf_out_adjust = Lerp(val0, val1, wgt, 3)
luma_lf_out_adjust = Clamp(luma_lf_out_adjust, 0, 1023)

```

3.5 Luma Freq Recompose

Input

Name	Type	Description
Luma lf out adjust	U10.0	Output low-frequency luma after global tone adjust.
Luma hf	S11.0	High-frequency component of luma.

Output

Name	Type	Description
Luma out	U10.0	Output luma.

Registers

Name	Type	Default Value	Description
ENABLE_FREQ_DECOMP	U1.0	True	Enable bit of luma frequency decomposition.

If ENABLE_FREQ_DECOMP=0, luma_out = Luma lf out. And if ENABLE_FREQ_DECOMP=1, luma_out is generated by summing the luma_lf_out and luma_hf input, as follows

$$\text{luma_out} = \text{clamp}(\text{luma_lf_out_adjust} + \text{luma_hf}, 0, 1023)$$

3.6 Color Mapping

This module maps the input RGB values based on both output luma and input luma, generating 12-bit output RGB

Input

Name	Type	Description
luma out	U10.0	Luma output calculate in freq recompose module.
luma in	U10.0	Luma value of input RGB.

rgb_input[3]	3*U10.0	Input RGB values.
--------------	---------	-------------------

Output

Name	Type	Description
rgb_output_12[3]	3*U12.0	Rgb out in color process module.

1. Ltm gain calculate

If luma_out == luma_in || luma_in == 0

ltm_gain = 1 << 12

Else

$$\text{ltm_gain} = \text{Clamp}\left(\frac{\text{luma_out} \ll 12 + \text{luma_in} \gg 1}{\text{luma_in}}, 0, (1 \ll 16) - 1\right)$$

$$\text{ltm_gain_max} = \text{Clamp}\left(\frac{1023 \ll 12}{\max(\text{R_in}, \text{G_in}, \text{B_in})}, 0, (1 \ll 16) - 1\right)$$

ltm_gain = min (ltm_gain, ltm_gain_max)

2. Rgb out calculate

R_out_12bit = **clamp**(RoundMutiply(ltm_gain, R, 10), 0,4095)

G_out_12bit = **clamp**(RoundMutiply(ltm_gain, G, 10), 0,4095)

B_out_12bit = **clamp**(RoundMutiply(ltm_gain, B, 10), 0,4095)

3.7 Dither

Input

Name	Type	Description
rgb_output_12[3]	3*U12.0	Rgb out after color process.

Output

Name	Type	Description
rgb_output_10[3]	3*U10.0	Final RGB output.

Registers

Name	Type	Default Value	Description
DITHER_TABLE_R[4]	4*U2.0	0,2,3,1	Dither threshold table of R
DITHER_TABLE_G[4]	4*U2.0	1,0,3,2	Dither threshold table of G

DITHER_TABLE_B[4]	4*U2.0	3,2,1,0	Dither threshold table of B
-------------------	--------	---------	-----------------------------

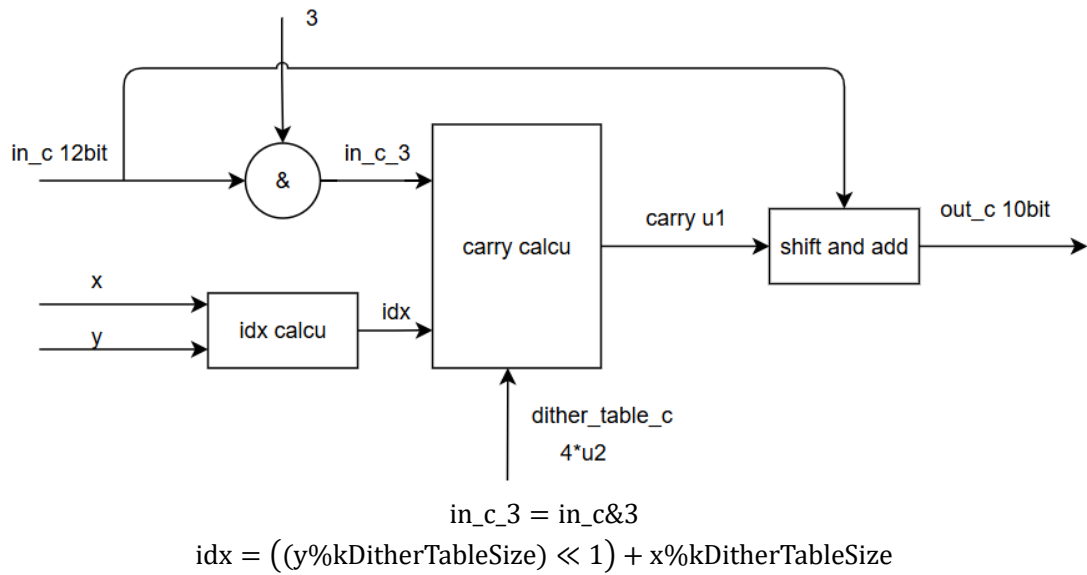
If ENABLE_DITHER is 0, rgb_output_12 will right 2bit with round to rgb_output_10.

```

rgb_output_10[0] = RoundShift(rgb_output_12[0],2)
rgb_output_10[1] = RoundShift(rgb_output_12[1],2)
rgb_output_10[2] = RoundShift(rgb_output_12[2],2)

```

Below is the block diagram of the module if ENABLE_DITHER is 1. We will explain it using one channel of the RGB three channels. In the figure, the input "in_c" of the module represents one of the RGB channels, x and y represent the coordinates of the current pixel, and dither_table_c represents the dither table for the current channel, and its values are configured by the software.



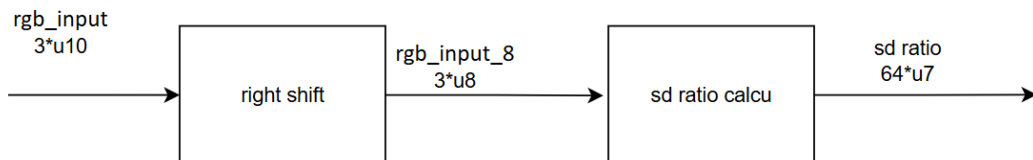
where $kDitherTableSize$ is 2.

```

threshold = dither_table_c[idx]
carry = { 1,  if in_c_3 > threshold
        { 0,  if in_c_3 ≤ threshold
out_c = clamp((in_c >> 2) + carry, 0, 1023)

```

3.8 Block avg sd



The module outputs the proportion of points with skin-tone characteristics in each block.

Input

Name	Type	Description
rgb_input	3*U10.0	Input rgb values.

Output

Name	Type	Description
sd_ratio	64*U7.0	Output block sd ratio.

Registers

Name	Type	Default Value	Description
SD_BLENDING_ENABLE	U1.0	True	Enable bit of scan detection.
BLOCK_RATIO_HR	U26.0		Norm coefficient of original resolution.
ROI_X_START	U12.0	0	X-coordinate of ROI start point.
ROI_Y_START	U12.0	0	Y-coordinate of ROI start point.
ROI_X_END	U12.0		X-coordinate of ROI end point(last point of ROI).
ROI_Y_END	U12.0		Y-coordinate of ROI end point(last point of ROI).
GRID_X_SCALE	U16.0		Norm coefficient of horizontal direction.
GRID_Y_SCALE	U16.0		Norm coefficient of vertical direction.
GRID_HEIGHT	U4.0	8	Grid numbers in vertical direction.
GRID_WIDTH	U4.0	8	Grid numbers in horizontal direction.
SD_R_THR	U8.0	95	R threshold of scan detection
SD_G_THR	U8.0	40	G threshold of scan detection
SD_B_THR	U8.0	20	B threshold of scan detection
SD_DELTA_MINMAX_TH R	U8.0	15	Delta_rgb threshold of scan detection

SD_DELTA_RG_THR	U8.0	15	Abs_rg threshold of scan detection
-----------------	------	----	------------------------------------

If SD_BLENDING_ENABLE=0, sd_ratio will be set to 0. If SD_BLENDING_ENABLE=1 the process of this module are as follows:

1. Define array: U24 sd_sum[64]={0}. Then Traverse the region of interest of the original image From point (ROI_X_START, ROI_Y_START) to point (ROI_X_END, ROI_Y_END)
2. for each pixel of step1, do follows operators:
 - a) Right-shifts the input RGB data by 2 bits(The reason is that the module operates on 8-bit data)
 - b) determine whether it belongs to a skin-tone region. A pixel is classified as a skin-tone point when it satisfies the following formula:

$$r > SD_R_THR \ \&\& \ g > SD_G_THR \ \&\& \ b > SD_B_THR \ \&\&$$

$$\text{delta} > SD_DELTA_MINMAX_THR \ \&\& \ \text{abs_rg} > SD_DELTA_RG_THR \ \&\& \ r > g \ \&\& \ r > b$$

where $\text{delta} = \text{Max}(r, g, b) - \text{Min}(r, g, b)$, $\text{abs_rg} = \text{abs}(r - g)$

- c) When a pixel is identified as a skin-tone point, its corresponding block is determined based on the pixel's x and y coordinates through the following calculation process:

$$\text{idx_x} = ((x - (\text{ROI_X_START})) * \text{GRID_X_SCALE}) \gg 16$$

$$\text{idx_y} = ((y - (\text{ROI_Y_START})) * \text{GRID_Y_SCALE}) \gg 16$$

$$\text{idx_x} = \text{clamp}(\text{idx_x}, 0, \text{GRID_WIDTH} - 1)$$

$$\text{idx_y} = \text{clamp}(\text{idx_y}, 0, \text{GRID_HEIGHT} - 1)$$

$$\text{idx} = \text{idx_x} + \text{idx_y} * 8$$

where ROI_X_START , ROI_Y_START, GRID_X_SCALE and GRID_Y_SCALE are configured via software.

- d) When a pixel is identified as a skin-tone point ,based on the index calculated in Step 3, increment the element at the corresponding position in sd_sum by one.

$$\text{sd_sum}[\text{idx}] += 1$$

3. Then Traverse grid:

idx_x from 0 to GRID_WIDTH-1, and idx_y from 0 to GRID_HEIGHT-1, for each idx_x and idx_y, calculates the block idx

$$\text{idx} = \text{idx_x} + \text{idx_y} * 8$$

$$\text{sum_idx} = \text{sd_sum}[\text{idx}]$$

$$\text{ratio_idx} = \text{RoundShift}(\text{sum_i} * 127 * \text{BLOCK_RATIO_HR}, 32)$$

$$\text{sd_ratio}[\text{idx}] = \text{Clamp}(\text{ratio_idx}, 0, 127)$$

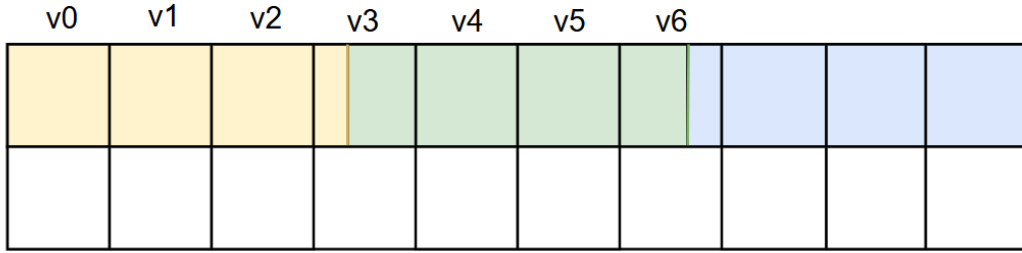
Where sd_ratio is the output of this module and will uploaded to the software.

3.9 Down Scale

Downscale reduces image size through downsampling, thereby decreasing computational costs associated with some feature statistics in algorithms. Downscale operates by averaging pixel values from a defined region in the source image (determined by the input-to-output size ratio) to generate each pixel in the reduced-resolution output.

First, let's take an input image with a width of 10 and downsample it to an output width of 3 as an example to explain the downsampling algorithm. Here, we focus solely on the image's width for clarity.

As shown in the figure below, we only consider the horizontal direction. Each black-bordered square represents a pixel in the original image, while the yellow, green, and blue colors represent the three output pixels after downsampling in the width direction. The output for the yellow region is obtained by averaging the original pixels it covers



Let's take the yellow section as an example to explain how horizontal downsampling is calculated. The calculation for the yellow section involves the four pixel values v0, v1, v2, and v3, but we cannot simply take their direct average as the output because v3 is not fully utilized (only partially covered by the downsampling window). For each row of pixels, we traverse from left to right. The weight assigned to each pixel equals the output width. During traversal, we accumulate these weights progressively. When the accumulated weight at a certain pixel exceeds the input width, we perform an output operation and reset the accumulated weight to zero. Below is a more intuitive explanation.

Weight average of the first output point are shown in the following equation:

$$\text{out}_0 = (v0 * 3 + v1 * 3 + v2 * 3 + v3 * (10 - 9))/10$$

In the above equation, the highlighted parts represent the weight of each pixel. For the first output point (yellow color):

For fully utilized pixels (v1, v2, v3): The weight is equal to the output width: 3, as these pixels are completely covered by the downsampling window.

For partially utilized pixels (v0): The weight is calculated as: input width - weight sum: $10 - (3 + 3 + 3)$

For the calculation of the second output point, the approach is similar, but weight of the first pixel (v3) is remaining usable weight is $3 - 1 = 2$

$$\text{out}_2 = (v3 * (3 - 1) + v4 * 3 + v5 * 3 + v6 * (10 - (3 - 1) - 3 - 3))/10$$

For the vertical direction, pixels from different rows also require weighted averaging. The process is similar to that of the horizontal direction. Please refer to the following pseudocode:

Input

Name	Type	Description
luma_in	U10.0	Input luma values.

Output

Name	Type	Description
luma_out	U10.0	Output down-scaled luma values.

Registers

Name	Type	Default Value	Description
ENABLE_DS	U1.0	1	Enable bit of downscale.
DS_OUTPUT_WIDTH	U12.0		Down-scaler output width.
DS_OUTPUT_HEIGHT	U12.0		Down-scaler output height.
DS_INPUT_WIDTH	U12.0		Down-scaler input width..
DS_INPUT_HEIGHT	U12.0		Down-scaler input height.
ROI_X_START	U12.0		X-coordinate of ROI start point.
ROI_Y_START	U12.0		Y-coordinate of ROI start point.
ROI_X_END	U12.0		X-coordinate of ROI end point(last point of ROI).
ROI_Y_END	U12.0		Y-coordinate of ROI end point(last point of ROI).
ROI_X_LEN	U12.0		Width of ROI.
ROI_Y_LEN	U12.0		Height of ROI.
DS_HOR_SCALE_NORM	U15.0		Norm coefficient of horizontal direction.
DS_VER_SCALE_NORM	U15.0		Norm coefficient of vertical direction.

```

Ver_weight_sum=0
Line_buff:  size: DS_OUTPUT_WIDTH*u20
Output_line:  size: DS_OUTPUT_WIDTH*u8

For y = ROI_Y_START;y<= ROI_Y_END;y++
    Current_line_weight=0
    Remained_line_weight=0
    Hor_weight_sum=0
    Hor_pixel_sum=0
    Ver_output=false
    Output_x_idx=0

    If(Ver_weight_sum+ DS_OUTPUT_HEIGHT< ROI_Y_LEN && y< ROI_Y_END):
        Current_line_weight = DS_OUTPUT_HEIGHT
        Remained_line_weight=0

    Else:
        Current_line_weight = ROI_Y_LEN - Ver_weight_sum
        remained_line_weight = DS_OUTPUT_HEIGHT - Current_line_weight
        ver_output=true

    For x = ROI_X_START;x<= ROI_X_END;x++
        If(hor_weight_sum+ DS_OUTPUT_WIDTH < ROI_X_LEN && x< ROI_X_END):
            Hor_pixel_sum += DS_OUTPUT_WIDTH*pixel_value
            Hor_weight_sum += DS_OUTPUT_WIDTH
        Else:
            Current_weight= ROI_X_LEN- Hor_weight_sum;
            Remained_weight = DS_OUTPUT_WIDTH- Current_weight
            Hor_pixel_sum += Current_weight *pixel_value
            Line_buff[output_x_idx]
                += RoundMultiply(Current_line_weight* Hor_pixel_sum ,
                    DS_HOR_SCALE_NORM, 21,20) .....Multiplier1

        If(ver_output):
            Output_line[output_x_idx]
                = clamp(RoundMultiply(Line_buff[output_x_idx], DS_VER_SCALE_NORM,21,10),0,1023)
                .....Multiplier2

            Line_buff[output_x_idx]
                = RoundMultiply(remain_line_weight* Hor_pixel_sum,
                    DS_HOR_SCALE_NORM, 21,20)
                .....Multiplier3

    output_x_idx++;

```

$\text{Hor_pixel_sum} = \text{Remained_weight} * \text{pixel_value}$
 $\text{Hor_weight_sum} = \text{Remained_weight}$

$\text{DS_HOR_SCALE_NORM} = (1 \ll 21) / \text{ROI_X_LEN}$
 $\text{DS_VER_SCALE_NORM} = (1 \ll 21) / \text{ROI_Y_LEN}$

The size of the line buffer required in this module is as follows:

$$u20 * 512 = 10240\text{bit}$$

The size of Multiplier in this module are as follows:

Multiplier1 $U30 * U15$
Multiplier2 $U20 * U15$
Multiplier3 $U30 * U15$

3.10 Block avg luma

This module outputs the average value of luma for each block, and the computation of this module is performed on the downscale image.

Input

Name	Type	Description
downscaled_luma	U8.0	downscaled luma values.

Output

Name	Type	Description
luma_avg	64*U8.0	Output average luma of each block.

Registers

Name	Type	Default Value	Description
LUMA_BLENDING_ENABLE	U1.0	True	Enable bit of average luma calculate.
BLOCK_RATIO_LR	U26.0		Norm coefficient of downscale resolution.
ENABLE_DS	U1.0	1	Enable bit of downscale.
HIST_GRID_X_SCALE	U16.0		Norm coefficient of horizontal direction.

HIST_GRID_Y_SCALE	U16.0		Norm coefficient of vertical direction.
GRID_HEIGHT	U4.0	8	Grid numbers in vertical direction.
GRID_WIDTH	U4.0	8	Grid numbers in horizontal direction.

If LUMA_BLENDING_ENABLE =0, this module will do nothing. If LUMA_BLENDING_ENABLE =1, the process of this module is as follows:

1. Define array: U32 luma_sum[64]={0}. Then traverse the downscale image. if ENABLE_DS=1, traverse the downscale image from start to end, and if ENABLE_DS=0, just traverse the ROI of downscale image.
2. for each pixel of step1, do follows operators:
 - a) Right-shifts the input downscaled_luma by 2 bits(The reason is that the module operates on 8-bit data)
 - b) Calculates the pixel's corresponding block determined based on the pixel's x and y coordinates through the following calculation process:

If ENABLE_DS is 0:

$$\text{idx_x} = ((x - \text{ROI_X_START}) * \text{HIST_GRID_X_SCALE}) \gg 16$$

$$\text{idx_y} = ((y - \text{ROI_Y_START}) * \text{HIST_GRID_Y_SCALE}) \gg 16$$

If ENABLE_DS is 1:

$$\text{idx_x} = (x * \text{HIST_GRID_X_SCALE}) \gg 16$$

$$\text{idx_y} = (y * \text{HIST_GRID_Y_SCALE}) \gg 16$$

$$\text{idx_x} = \text{clamp}(\text{idx_x}, 0, \text{GRID_WIDTH} - 1)$$

$$\text{idx_y} = \text{clamp}(\text{idx_y}, 0, \text{GRID_HEIGHT} - 1)$$

$$\text{idx} = \text{idx_x} + \text{idx_y} * 8$$

where HIST_GRID_X_SCALE, HIST_GRID_Y_SCALE are configured via software. This module's pixel-to-block calculation shares logic with the histogram statistics module (except for the frac computation). You can optionally reuse the histogram's calculation process.

- c) When a pixel is identified as a skin-tone point ,based on the index calculated in Step 3, increment the luma_sum at the corresponding position by luma value of the pixel.

$$\text{luma_sum}[\text{idx}] += \text{luma}$$

where luma_in is the luma value of the pixel.

3. Then Traverse grid:

idx_x from 0 to GRID_WIDTH, and idx_y from 0 to GRID_HEIGHT, for each idx_x and idx_y, calculates the block idx

$$\text{idx} = \text{idx_x} + \text{idx_y} * 8$$

$$\text{sum_idx} = \text{luma_sum}[\text{idx}]$$

$$\text{avg_idx} = \text{RoundShift}(\text{sum_idx} * \text{BLOCK_RATIO_LR}, 32)$$

$$\text{luma_avg}[\text{idx}] = \text{clamp}(\text{avg_idx}, 0, 255)$$

Where luma_avg is the output of this module and will uploaded to the software.

3.11 Block avg wopr

The module calculates the proportion of points within each block where the luma value exceeds the threshold, the threshold is configured via software.

Input

Name	Type	Description
Downscaled_luma	U8.0	Input luma values.

Output

Name	Type	Description
Wopr_avg	64*U7.0	Output white point ratio of each block.

Registers

Name	Type	Default Value	Description
WOPR_BLENDING_ENABLE	U1.0	True	Enable bit of wopr module.
BLOCK_RATIO_LR	U26.0		Norm coefficient of downscale resolution.
ENABLE_DS	U1.0	1	Enable bit of downscale.
HIST_GRID_X_SCALE	U16.0		Norm coefficient of horizontal direction.
HIST_GRID_Y_SCALE	U16.0		Norm coefficient of vertical direction.
WOPR_THR	U8.0		Threshold of luma.
GRID_HEIGHT	U4.0	8	Grid numbers in vertical direction.
GRID_WIDTH	U4.0	8	Grid numbers in horizontal direction.

If WOPR_BLENDING_ENABLE =0, this module will do nothing. If WOPR_BLENDING_ENABLE =1 the process of this module are as follows:

1. Define array: U24 wopr_sum[64]={0}. Then traverse the downscale image. if ENABLE_DS=1, traverse the downscale image from start to end, and if ENABLE_DS=0, just traverse the ROI of downscale image.

2. for each pixel of step1, do follows operators:
 - a) Right-shifts the luma_in by 2 bits(The reason is that the module operates on 8-bit data)
 - b) determine whether it belongs to white point. A pixel is classified as a white point when its luma value large than WOPR_THR
 if luma_in > WOPR_THR, the point is white point
 - c) When a pixel is identified as a white point, its corresponding block is determined based on the pixel's x and y coordinates through the following calculation process:

If ENABLE_DS is 0:

$$\text{idx_x} = ((x - \text{ROI_X_START}) * \text{HIST_GRID_X_SCALE}) \gg 16$$

$$\text{idx_y} = ((y - \text{ROI_Y_START}) * \text{HIST_GRID_Y_SCALE}) \gg 16$$

If ENABLE_DS is 1:

$$\text{idx_x} = (x * \text{HIST_GRID_X_SCALE}) \gg 16$$

$$\text{idx_y} = (y * \text{HIST_GRID_Y_SCALE}) \gg 16$$

$$\text{idx_x} = \text{clamp}(\text{idx_x}, 0, \text{GRID_WIDTH} - 1)$$

$$\text{idx_y} = \text{clamp}(\text{idx_y}, 0, \text{GRID_HEIGHT} - 1)$$

$$\text{idx} = \text{idx_x} + \text{idx_y} * 8$$

where HIST_GRID_X_SCALE, HIST_GRID_Y_SCALE are configured via software. This module's pixel-to-block calculation shares logic with the histogram statistics module (except for the frac computation). You can optionally reuse the histogram's calculation process.

- d) When a pixel is identified as a **wopr** point ,based on the index calculated in Step 3, increment the element at the corresponding position in sd_sum by one.

$$\text{wopr_sum}[\text{idx}] += 1$$

3. Then Traverse grid:

idx_x from 0 to GRID_WIDTH, and idx_y from 0 to GRID_HEIGHT, for each idx_x and idx_y, calculates the block idx

$$\text{idx} = \text{idx_x} + \text{idx_y} * 8$$

$$\text{sum_idx} = \text{wopr_sum}[\text{idx}]$$

$$\text{ratio_idx} = \text{RoundShift}(\text{sum_idx} * 127 * \text{BLOCK_RATIO_LR}, 32)$$

$$\text{wopr_avg}[\text{idx}] = \text{Clamp}(\text{ratio_idx}, 0, 127)$$

Where wopr_avg is the output of this module and will uploaded to the software.

3.12 Content Detection

The purpose of content detection is to determine whether a pixel is located in a detailed area. If it is, the pixel will be assigned a greater weight during histogram statistics.

Input

Name	Type	Description
luma	U10.0	Input luma values.

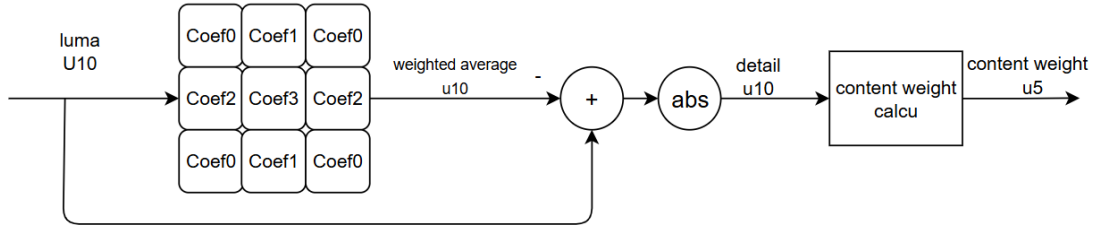
Output

Name	Type	Description
Content wgt	U5.0	The weight of pixels in histogram computation.

Registers

Name	Type	Default Value	Description
ENABLE_CONTENT_DETECTION	U1.0	1	Enable bit of content detection
CONTENT_DETECTION_COEF0	U8.0	8	Content detection filter coefficient 0.
CONTENT_DETECTION_COEF1	U8.0	16	Content detection filter coefficient 1.
CONTENT_DETECTION_COEF2	U8.0	16	Content detection filter coefficient 2.
CONTENT_DETECTION_COEF3	U8.0	32	Content detection filter coefficient 3.
CONTENT_DETECTION_THRESHOLD0	U10.0	4	Content detection threshold coefficient 0.
CONTENT_DETECTION_THRESHOLD1	U10.0	48	Content detection threshold coefficient 1.
CONTENT_DETECTION_THRESHOLD2	U10.0	256	Content detection threshold coefficient 2.
CONTENT_DETECTION_THRESHOLD3	U10.0	512	Content detection threshold coefficient 3.
CONTENT_DETECTION_SLOPE0	U1.14	372	Content detection slope coefficient 0.
CONTENT_DETECTION_SLOPE1	U1.14	64	Content detection slope coefficient 1.

CONTENT_DETECTION_MIN_WGT	U2.0	2	Luma frequency decomposition filter coefficient 5.
---------------------------	------	---	--



If the module is not enable(ENABLE_CONTENT_DETECTION==0),content weight is set kContentDetectionWgtOne(value is 16). If the module is enable, The calculation process of this module is as follows.

Weighted average

Compute the weighted average within a 3x3 window centered on the current pixel, where the weights are determined by the register-configured CONTENT_DETECTION_COEF. As shown in the following equation:

$$LUMA = \begin{pmatrix} luma_{[(i-1)(j-1)]} & luma_{[(i-1)j]} & luma_{[(i-1)(j+1)]} \\ luma_{[i(j-1)]} & luma_{[ij]} & luma_{[i(j+1)]} \\ luma_{[(i+1)(j-1)]} & luma_{[(i+1)j]} & luma_{[(i+1)(j+1)]} \end{pmatrix}$$

$$COEF = \begin{pmatrix} CONTENT_DETECTION_COEF_0 & CONTENT_DETECTION_COEF_1 & CONTENT_DETECTION_COEF_0 \\ CONTENT_DETECTION_COEF_2 & CONTENT_DETECTION_COEF_3 & CONTENT_DETECTION_COEF_2 \\ CONTENT_DETECTION_COEF_0 & CONTENT_DETECTION_COEF_1 & CONTENT_DETECTION_COEF_0 \end{pmatrix}$$

$$weight_sum = SumMultiply(LUMA, COEF)$$

Where, SumMultiply first performs element-wise multiplication of the two input parameters (matrices), then sums up the resulting values.

The sum of the CONTENT_DETECTION_COEF configured by the software algorithm is 128, so it can directly right shift weight_sum to obtain the weight_average, as shown in the following formula:

$$weight_average = (weight_sum + 64) \gg 8$$

Detail calcu

$$detail = abs(luma - weight_average)$$

content weight calcu

define follow function

LUT1D(input, th0, th1, slope, wgt_max) is

$$\begin{aligned} &\text{when input} < th0, \text{ output} = 0 \\ &\text{when input} \geq th1, \text{ output} = wgt_max \\ &\text{else output} = \text{clamp}(\text{RoundShift}((\text{input} - th0) * \text{slope} * wgt_max, \text{slope_frac}), 0, wgt_max) \end{aligned}$$

Where round is rounding function with slope_frac precision, and the value of slope_frac is 14

The content weight calculate is as follows:

wgt0
= LUT1D(detail, CONTENT_DETECTION_THRESHOLD0, CONTENT_DETECTION_THRESHOLD1,
CONTENT_DETECTION_SLOPE0, kContentDetectionWgtOne

wgt1 = kContentDetectionWgtOne – LUT1D(detail, CONTENT_DETECTION_THRESHOLD2,
CONTENT_DETECTION_THRESHOLD3, CONTENT_DETECTION_SLOPE1, kContentDetectionWgtOne)

content weight = MAX(MIN(wgt0, wgt1), CONTENT_DETECTION_MIN_WGT)

where kContentDetectionWgtOne is equal to 16, and detail is the value we calculated above,
others are register parameter.

3.13 Local Histogram

The module performs histogram statistics on the downscale luma values of the image and reports the statistical results to the software.

Input

Name	Type	Description
Luma	U10.0	Input luma values.
Content_weight	U5.0	Content_weight of a pixel.

Output

Name	Type	Description
HISTOGRAM	64*64*U32.0	The weight of pixels in histogram computation.

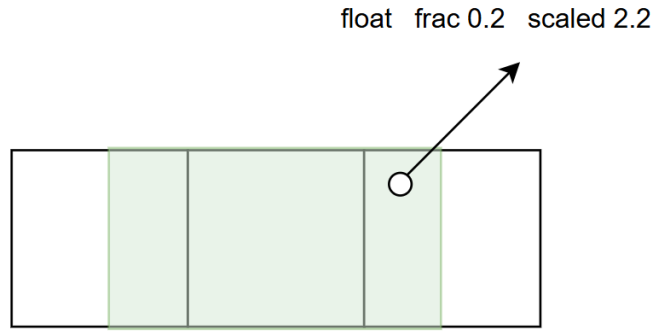
Registers

Name	Type	Default Value	Description
------	------	---------------	-------------

ENABLE_HIST_OVERLAP	U1.0	1	Enable bit of histogram overlap.
ENABLE_HIST	U1.0	1	Enable bit of histogram statistics.
ROI_X_START	U12.0		X-coordinate of ROI start point.
ROI_Y_START	U12.0		Y-coordinate of ROI start point.
ROI_X_END	U12.0		X-coordinate of ROI end point(last point of ROI).
ROI_Y_END	U12.0		Y-coordinate of ROI end point(last point of ROI).
ROI_X_LEN	U12.0		Width of ROI.
ROI_Y_LEN	U12.0		Height of ROI.
HIST_GRID_X_SCALE	U16.0		Norm coefficient of horizontal direction.
HIST_GRID_Y_SCALE	U16.0		Norm coefficient of vertical direction.
HIST_GRID_DEPTH	U7.0	64	Histogram bin numbers.
HIST_OVERLAP_RATIO	U0.10		Overlap ratio during histogram statistics.
ENABLE_DS	U1.0		Enable bit of downscale.
ENABLE_CONTENT_DETECTION	U1.0	1	Enable bit of content detection
GRID_HEIGHT	U4.0	8	Grid numbers in vertical direction.
GRID_WIDTH	U4.0	8	Grid numbers in horizontal direction.

If ENABLE_HIST is 0,this module will do nothing. If ENABLE_HIST is 1, The process of this module is as follows:

1. Traverse the the downscale image
If ENABLE_DS is 0, the region of interest in the downscale image(original image) will be traversed. If ENABLE_DS is 1, the whole downscale image will be traversed.
2. for each pixel of step1, do follows operators:
 - a) Based on the coordinates (x,y) of each pixel, calculate the block it belongs to, the flag and the neighbor, where the flag is used to mark whether the point needs to be counted in other histograms. The specific process is as follows:



If ENABLE_DS is 0:

$$\text{scaled_x} = (x - \text{ROI_X_START}) * \text{HIST_GRID_X_SCALE}$$

$$\text{scaled_y} = (y - \text{ROI_Y_START}) * \text{HIST_GRID_Y_SCALE}$$

If ENABLE_DS is 1:

$$\text{scaled_x} = x * \text{HIST_GRID_X_SCALE}$$

$$\text{scaled_y} = y * \text{HIST_GRID_Y_SCALE}$$

The remaining calculations are independent of the value of ENABLE_DS

$$\text{index_x} = \text{sclae_x} \gg 16$$

$$\text{frac} = (\text{scaled_x} - (\text{index_x} \ll 16)) \gg 6$$

$$\text{index_x} = \text{clamp}(\text{index_x}, 0, \text{GRID_WIDTH} - 1)$$

$$\text{if}(\text{frac} > 1024 - \text{HIST_OVERLAP_RATIO})$$

$$\text{neighbor} = \text{clamp}(\text{index_x} + 1, 0, 7)$$

$$\text{flag} = 1$$

$$\text{else if}(\text{frac} < \text{HIST_OVERLAP_RATIO})$$

$$\text{neighbor} = \text{clamp}(\text{index_x} - 1, 0, 7)$$

$$\text{flag} = 1$$

$$\text{else } \text{flag} = 0$$

$$\text{index_y} = \text{sclae_y} \gg 16$$

$$\text{index_y} = \text{clamp}(\text{index_y}, 0, \text{GRID_HEIGHT} - 1)$$

$$\text{index_z} = (\text{luma} * \text{HIST_GRID_DEPTH}) \gg 10$$

$$\text{index_z} = \text{clamp}(\text{z_index}, 0, \text{HIST_GRID_DEPTH} - 1)$$

where luma is the luma value of the pixel.

3. Perform histogram statistics based on the value of a and the index calculated in the second step.

$$\text{hist_val} = \text{ENABLE_CONTENT_DETECTION? } \text{content_weight: } 1$$

where content_weight is the content_weight of the pixel(calculated in Content

Detection). ENABLE_CONTENT_DETECTION is configured via software.

$$\begin{aligned} \text{hist_index} &= \text{index_x} + \text{index_y} * 8 \\ \text{sum} &= \text{HISTOGRAM}[\text{hist_index}][\text{index_z}] + \text{hist_val} \\ \text{HISTOGRAM}[\text{hist_index}][\text{index_z}] &= \text{clamp}(\text{sum}, 0, (1 \ll 32) - 1) \end{aligned}$$

if flag = 1(flag was calculated in step 2) and ENABLE_HIST_OVERLAP=1, then the pixel will be counted not only in its own block when calculating the histogram, but also in the neighboring blocks, as follows:

$$\begin{aligned} \text{hist_index_neighbor} &= \text{neighbor} + \text{index_y} * 8 \\ \text{sum} &= \text{HISTOGRAM}[\text{hist_index_neighbor}][\text{index_z}] + \text{hist_val} \\ \text{HISTOGRAM}[\text{hist_index_neighbor}][\text{index_z}] &= \text{clamp}(\text{sum}, 0, (1 \ll 32) - 1) \end{aligned}$$

After all pixels have been traversed, HISTOGRAM is reported to the software.

3.14 ROI detection

Input

Name	Type	Description
luma	U10.0	Input luma values.

Output

Name	Type	Description
Sum roi	U24.0	Number of text point.

Registers

Name	Type	Default Value	Description
AD_TXT_JUD_EN	U1.0	1	Enable bit of roi detection

This module is used to determine whether the current image contains text.

If AD_TXT_JUD_EN==0, sum roi =0, if AD_TXT_JUD_EN==1, The operation are as follows

When all pixels within a 3*5 window centered on the current point have equal luma values, we classify that point as a text point. The module counts the number of text points in the current image and reports the result to the software. Pseudo code are as follows, while luma_win_c is the luma value of the center pixel of the window:

Sum_roi=0

Traverse the whole image:

Flag=1

```

Traverse the window:
    If(luma_win==luma_win_c):
        Continue
    Else:
        Flag=0
        Break
If(flag==1):
    Sum_roi+=1

```

3.15 Block avg texture

Input

Name	Type	Description
luma	U10.0	Input luma values.

Output

Name	Type	Description
avg_texture	64*U8.0	Output average texture of each block.

Registers

Name	Type	Default Value	Description
TEX_BLENDING_ENABLE	U1.0	1	Enable bit of average texture
BLOCK_RATIO_HR	U26.0		Norm coefficient of original resolution.
TEX_DIFF_LUT	33*U9.0	1	Texture diff lut to calculate texture feature
ROI_X_START	U12.0	0	X-coordinate of ROI start point.
ROI_Y_START	U12.0	0	Y-coordinate of ROI start point.
ROI_X_END	U12.0		X-coordinate of ROI end point(last point of ROI).
ROI_Y_END	U12.0		Y-coordinate of ROI end point(last point of ROI).
GRID_X_SCALE	U16.0		Norm coefficient of horizontal direction.
GRID_Y_SCALE	U16.0		Norm coefficient of vertical direction.

GRID_HEIGHT	U4.0	8	Grid numbers in vertical direction.
GRID_WIDTH	U4.0	8	Grid numbers in horizontal direction.

If `TEX_BLENDING_ENABLE==0`, `avg_texture=0`, if `TEX_BLENDING_ENABLE==1`, operation are as follows:

1. Define array: `U32 texture_sum[64]={0}`. Then traverse the ROI of original resolution luma image.
2. for each pixel of step1, do follows operators:
 - a) Right-shifts the `luma_in` by 2 bits(The reason is that the module operates on 8-bit data)
 - b) Compute the `texture_diff` for each pixel as follows:

$$\text{texture_diff} = \text{clamp}\left(\left\{\sum \text{abs}(\text{luma_win} - \text{luma_win_c})\right\} * 18\right) \gg 6, 0, 255$$

In the above formula, the summation is performed over the pixels within the 3×5 window centered on the current pixel. `luma_win_c` is the luma value of the current pixel, `luma_win` are luma value of other pixel in the window.

- c) After obtaining the texture diff for each pixel, it is passed through a lookup table (LUT) to derive the texture feature for each pixel point. As follows:

$$\begin{aligned} \text{idx0} &= \text{texture_diff} \gg 3 \\ \text{idx1} &= \text{clamp}(\text{idx0} + 1, 0, 32) \\ \text{wgt} &= \text{texture_diff} - (\text{idx0} \ll 3) \\ \text{val0} &= \text{TEX_DIFF_LUT}[\text{idx0}] \\ \text{val1} &= \text{TEX_DIFF_LUT}[\text{idx1}] \\ \text{texture_feature} &= \text{Lerp}(\text{val0}, \text{val1}, \text{wgt}, 3) \end{aligned}$$

- d) When a pixel is identified as a white point, its corresponding block is determined based on the pixel's x and y coordinates through the following calculation process:

$$\text{idx_x} = \left((x - (\text{ROI_X_START})) * \text{GRID_X_SCALE}\right) \gg 16$$

$$\text{idx_y} = \left((y - (\text{ROI_Y_START})) * \text{GRID_Y_SCALE}\right) \gg 16$$

$$\begin{aligned} \text{idx_x} &= \text{clamp}(\text{idx_x}, 0, \text{GRID_WIDTH} - 1) \\ \text{idx_y} &= \text{clamp}(\text{idx_y}, 0, \text{GRID_HEIGHT} - 1) \\ \text{idx} &= \text{idx_x} + \text{idx_y} * 8 \end{aligned}$$

where `HIST_GRID_X_SCALE`, `HIST_GRID_Y_SCALE` are configured via software. This module's pixel-to-block calculation shares logic with the histogram statistics module (except for the `frac` computation). You can optionally reuse the histogram's calculation process.

$\text{texture_sum}[\text{idx}] += \text{texture_feature}$

3. Then Traverse grid:

idx_x from 0 to GRID_WIDTH, and idx_y from 0 to GRID_HEIGHT, for each idx_x and idx_y, calculates the block idx

$\text{idx} = \text{idx_x} + \text{idx_y} * 8$

$\text{texture_idx} = \text{RoundShift}(\text{texture_sum}[\text{idx}] * \text{BLOCK_RATIO_HR}, 32)$

$\text{avg_texture}[\text{idx}] = \text{Clamp}(\text{texture_idx}, 0, 255)$

Where avg_texture is the output of this module and will uploaded to the software.